```
// BootRunner.cs
using System.Collections;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using Logger = Sy.Core.Logger;
namespace Sy.P07.Boot
{
    public static class SceneIds
    {
        public const string Boot = "S00_Boot";
        public const string Main = "S07_Main";
    }
    /// &lt;summary&gt;
    /// S00_Boot       Mock  →   →     T001④
    ///      Boot            UI T001b  Boot      =
    /// &lt;/summary&gt;
    public sealed class BootRunner : MonoBehaviour
    {
        IEnumerator Start()
        {
            BootServices.RegisterAll();
            var sequence = new BootSequence(SwitchToMain);
            if (!sequence.RunAll())
            {
                Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                yield break;
            }
            yield return null;
        }
        bool SwitchToMain()
        {
            StartCoroutine(LoadMainRoutine());
            return true;
        }
        IEnumerator LoadMainRoutine()
        {
            var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
            while (op != null &amp;&amp; !op.isDone) yield return null;
        }
    }
}
// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
namespace Sy.P07.Boot
{
    /// &lt;summary&gt;
```

```
/// Boot     T001
/// ①Logger→②  →③Config→④Save→⑤ILocale→⑥Ads.Preload→⑦         [BOOT] n/7
///  ⑦        Play  =SceneManager.LoadSceneAsync Editor dry-run=EditorSceneManager.OpenScene
/// X006      SY_MOCK
/// &lt;/summary&gt;
public sealed class BootSequence
{
    public const int TotalSteps = 7;
    readonly Func&lt;bool&gt; _switchScene;
    readonly List&lt;string&gt; _lines = new();
    public BootSequence(Func&lt;bool&gt; switchScene)
    {
        _switchScene = switchScene;
    }
    public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
    public bool RunAll()
    {
        bool ok = true;
        ok &amp;= Run(1, "Logger", LoggerStep);
        ok &amp;= Run(2, "Time", TimeStep);
        ok &amp;= Run(3, "Config", ConfigStep);
        ok &amp;= Run(4, "Save", SaveStep);
        ok &amp;= Run(5, "ILocale", LocaleStep);
        ok &amp;= Run(6, "Ads.Preload", AdsStep);
        ok &amp;= Run(7, "SwitchMain", () =&gt; _switchScene != null &amp;&amp; _switchScene());
        return ok;
    }
    bool Run(int n, string name, Func&lt;bool&gt; step)
    {
        bool result;
        try
        {
            result = step();
        }
        catch (Exception e)
        {
            Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
            result = false;
        }
        string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
        _lines.Add(line);
        Logger.Info("BOOT", line);
        return result;
    }
    static bool LoggerStep() =&gt; true; //              JS sink platform T001c
    static bool TimeStep()
    {
        var time = ServiceLocator.Get&lt;ITimeService&gt;();
        return time.ServerNowMs() &gt; 0 &amp;&amp; !time.TimeCheated();
    }
```

```
        static bool ConfigStep()
        {
            var config = ServiceLocator.Get&lt;IConfigService&gt;();
            var update = config.CheckUpdate("mock://manifest");
            return update.NewVersion != null; // mock=    X009   manifest
        }
        static bool SaveStep()
        {
            var save = ServiceLocator.Get&lt;ISaveService&gt;();
            save.Save("boot_probe", "{}");
            return !string.IsNullOrEmpty(save.Load("boot_probe"));
        }
        static bool LocaleStep()
        {
            var locale = ServiceLocator.Get&lt;ILocaleService&gt;();
            return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont); //         +
        }
        static bool AdsStep()
        {
            ServiceLocator.Get&lt;IAdsService&gt;().Preload("boot_warmup");
            return true;
        }
    }
}
// BootServices.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Core;
using Sy.Platform;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.P07.Boot
{
    /// &lt;summary&gt;
    ///    Mock    SY_MOCK    X006     Mock X002a   AppID touristappid
    /// T001c  WX/TT     Channel   Editor      Boot
    /// &lt;/summary&gt;
    public static class BootServices
    {
        public static void RegisterAll()
        {
            if (ServiceLocator.TryGet&lt;ITimeService&gt;(out _)) return; //
            ServiceLocator.Register&lt;ITimeService&gt;(new MockTimeService());
            ServiceLocator.Register&lt;IConfigService&gt;(new MockConfigService());
            ServiceLocator.Register&lt;ISaveService&gt;(new MockSaveService());
            ServiceLocator.Register&lt;ILocaleService&gt;(new MockLocaleService());
            ServiceLocator.Register&lt;IAdsService&gt;(new MockAdsService());
            Logger.Info("BOOT", "mock services registered (SY_MOCK)");
        }
```

```
}
public sealed class MockTimeService : ITimeService
{
    public TimeSource Source =&gt; TimeSource.Mock;
    public long ClientNowMs() =&gt; DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    /// &lt;summary&gt;SY_MOCK X006                  TCB serverTime→HTTPS Date →   T001b   &lt;/summary&gt;
    public long ServerNowMs() =&gt; ClientNowMs();
    public bool TimeCheated() =&gt; Math.Abs(ServerNowMs() - ClientNowMs()) &gt; 120_000;
}
public sealed class MockConfigService : IConfigService
{
    public T GetTable&lt;T&gt;(string tableId) where T : class
        =&gt; throw new KeyNotFoundException("mock config table not loaded: " + tableId);
    public bool TryGetTable&lt;T&gt;(string tableId, out T table) where T : class
    {
        table = null;
        return false;
    }
    public ConfigUpdateResult CheckUpdate(string manifestUrl)
        =&gt; new ConfigUpdateResult(false, "mock-0.0.1");
    public void SwapPolicy(ConfigSwapPolicy policy) { }
    public string PinConfigVersion(string scope) =&gt; "mock-0.0.1";
}
public sealed class MockSaveService : ISaveService
{
    const string Prefix = "sy_save_";
    public void Save(string slot, string payloadJson) =&gt; PlayerPrefs.SetString(Prefix + slot, payloadJson);
    public string Load(string slot) =&gt; PlayerPrefs.GetString(Prefix + slot, string.Empty);
    /// &lt;summary&gt;X006             T001b v1→v2     &lt;/summary&gt;
    public SyncResult CloudSync() =&gt; new SyncResult(false, null);
}
public sealed class MockLocaleService : ILocaleService
{
    public void SetLang(string lang) { }
    /// &lt;summary&gt; key  key      T001b cfg_i18n +  NUnit    &lt;/summary&gt;
    public string GetText(string key, params string[] args)
    {
        Logger.Warn("LOCALE", "missing i18n key (fallback): " + key);
        return key;
    }
    public FontSet GetFontSet()
        =&gt; new FontSet("LegacyRuntime.ttf", "LegacyRuntime.ttf", "LegacyRuntime.ttf");
}
public sealed class MockAdsService : IAdsService
{
    public event Action&lt;AdLifecycle&gt; OnAdEvent;
    public Task&lt;RewardResult&gt; ShowRewarded(string pointId)
        =&gt; Task.FromResult(new RewardResult { Completed = true, Reason = "mock" });
    public void ShowInterstitial(string pointId) { }
    public void ShowBanner(string pointId) { }
```

```
        public void Preload(string pointId)
        {
            Logger.Info("ADS", "preload (mock): " + pointId);
            OnAdEvent?.Invoke(new AdLifecycle(AdLifecycleType.Fill, pointId, "mock"));
        }
    }
}
// MainUI.cs
using UnityEngine;
using UnityEngine.SceneManagement;
namespace Sy.P07.Newbie
{
    public static class NewbieSceneIds
    {
        public const string Main = "S07_Main";
        public const string Battle = "S07_Battle";
    }
    /// &lt;summary&gt;S07          →S07_Battle M5      ——           &lt;/summary&gt;
    public sealed class MainUI : MonoBehaviour
    {
        public void StartRestoration()
        {
            SceneManager.LoadScene(NewbieSceneIds.Battle);
        }
        void Awake()
        {
            Application.targetFrameRate = 30;
            Sy.Core.MainArtInjector.Apply(transform, "P07_scene_01", "P07_unit_01");
        }
    }
}
// BattleLauncher.cs
using System.Collections.Generic;
using UnityEngine;
namespace Sy.P07.Newbie
{
    /// &lt;summary&gt;
    /// S07_Battle       MuseumSim+TutorialFlow+MuseumView   Tick
    ///    Sy.Telemetry Editor =Debug sink T001e        +TCB
    /// HUD                     6
    /// &lt;/summary&gt;
    public sealed class BattleLauncher : MonoBehaviour
    {
        MuseumSim _sim;
        TutorialFlow _tutorial;
        MuseumView _view;
        Sy.Telemetry.TelemetryService _telemetry;
        int _lastEnergy = -1, _lastFragments = -1, _lastSeals = -1;
        void Awake()
        {
```

```
    Application.targetFrameRate = 30;
    var canvas = FindObjectOfType&lt;Canvas&gt;();
    _sim = new MuseumSim(NewbieConfig.SimSeed);
    _view = gameObject.AddComponent&lt;MuseumView&gt;();
    _view.Initialize(canvas, _sim);
    _telemetry = new Sy.Telemetry.TelemetryService();
    _telemetry.AddSink(new TelemetryDebugSink());
    _tutorial = new TutorialFlow(
        _sim,
        hint =&gt;
        {
            Debug.Log("[HINT] " + hint);
            _view.SetHint(hint);
        },
        step =&gt; _telemetry.Track(Sy.Telemetry.EventIds.NewbieStep,
            new Dictionary&lt;string, string&gt; { { "step", step.ToString() } }));
    _view.CellTapped += OnCellTapped;
    _view.DigRequested += OnDigRequested;
    _view.DeliverRequested += OnDeliverRequested;
    _view.DecomposeRequested += OnDecomposeRequested;
    _view.RelayTapped += OnRelayTapped;
    _sim.Dug += OnDug;
    _sim.Merged += OnMerged;
    _sim.Decomposed += OnDecomposed;
}
void Update()
{
    _sim.Tick(Time.deltaTime);
    _view.SyncView(_sim);
    if (_sim.Energy != _lastEnergy) { _lastEnergy = _sim.Energy; _view.SetEnergy(_lastEnergy); }
    if (_sim.FragmentCount != _lastFragments) { _lastFragments = _sim.FragmentCount; _view.SetFragments(_lastFra
    if (_sim.SealCount != _lastSeals) { _lastSeals = _sim.SealCount; _view.SetSeals(_lastSeals); }
}
void OnCellTapped(int col, int row)
{
    _sim.SelectOrMerge(col, row);
}
void OnDigRequested()
{
    _sim.TryDig();
}
void OnDeliverRequested()
{
    _sim.DeliverOrder();
}
void OnDecomposeRequested()
{
    _sim.DecomposeBronze();
}
void OnRelayTapped()
```

```
        {
            _view.SetHint("      ——      1                  ");
        }
        void OnDug()
        {
            _view.PlayDigFx();
        }
        void OnMerged(MuseumSim.ItemInst a, MuseumSim.ItemInst b, MuseumSim.ItemInst product)
        {
            _view.PlayMergeFx(product.Id);
        }
        void OnDecomposed(MuseumSim.ItemInst bronze)
        {
            _view.PlayDecomposeFx();
        }
        sealed class TelemetryDebugSink : Sy.Telemetry.TelemetryService.ISink
        {
            public void Write(string evtId, Dictionary&lt;string, string&gt; fields)
            {
                var sb = new System.Text.StringBuilder(evtId);
                foreach (var kv in fields)
                {
                    sb.Append(' ').Append(kv.Key).Append('=').Append(kv.Value);
                }
                Debug.Log("[TELEMETRY] " + sb);
            }
        }
    }
}
// MuseumSim.cs
using System;
using System.Collections.Generic;
namespace Sy.P07.Newbie
{
    /// &lt;summary&gt;
    ///              C#       Play   Editor headless   NewbieFlowCheck
    ///          §4.2      ①         →  →       2 1
    /// ②         +          →   1    2  ="         "
    /// ③       3        +   /NPC  +
    ///      06  07   M0    3 →M1     -1·    ·     →M2     + =
    /// →M2.5      + =    →    →        →M3      3  →
    /// →M4   #1+NPC    →M5      +    +      +
    ///               3          3 ——
    /// &lt;/summary&gt;
    public sealed class MuseumSim
    {
        public enum MuseumPhase { Journal1, Journal2, Journal3, Playing, CardShow, WonderPreview, Settled }
        public sealed class ItemInst
        {
            public int Id;
```

```
    public int TypeId;    // 0  /1  /2
    public int Col, Row;
}
public MuseumPhase Phase { get; private set; } = MuseumPhase.Journal1;
readonly List&lt;ItemInst&gt; _items = new();
readonly Random _rng;
int _nextId = 1;
float _phaseTimer;
int _selectedId = -1;
public IReadOnlyList&lt;ItemInst&gt; Items =&gt; _items;
public int Energy { get; private set; } = NewbieConfig.StartEnergy;
public int SealCount { get; private set; } = NewbieConfig.StartSeals;
public int DigCount { get; private set; }
public int MergeCount { get; private set; }
public int MisMergeCount { get; private set; }     //       +   ——
public int DecomposeCount { get; private set; }     //
public int FragmentCount { get; private set; }
public int BronzeCount { get; private set; }
public bool OrderDelivered { get; private set; }
public bool WonderUnlocked { get; private set; }
public bool RelayUnlocked { get; private set; }
public int SelectedItemId =&gt; _selectedId;
public float PhaseTimer =&gt; _phaseTimer;
public float RegionProgress { get; private set; } = NewbieConfig.RegionProgressStart;
public int AdPopupCount =&gt; 0;                        //
public event Action&lt;int&gt; EnergyChanged;
public event Action&lt;int&gt; SealChanged;
public event Action Dug;                                    //          -1·
public event Action&lt;ItemInst, ItemInst, ItemInst&gt; Merged;    // (A, B,   )
public event Action&lt;ItemInst&gt; MisMerged;                    //      =    ——
public event Action&lt;ItemInst&gt; Decomposed;                   //           →2
public event Action OrderDeliveredDone;
public event Action CardShown;
public event Action WonderPreviewShown;
public event Action Settled;
public event Action&lt;MuseumPhase&gt; PhaseChanged;
public MuseumSim(int seed)
{
    _rng = new Random(seed);
}
public void Tick(float dt)
{
    _phaseTimer += dt;
    switch (Phase)
    {
        case MuseumPhase.Journal1:
            if (_phaseTimer &gt;= NewbieConfig.JournalPageSeconds) SetPhase(MuseumPhase.Journal2);
            break;
        case MuseumPhase.Journal2:
            if (_phaseTimer &gt;= NewbieConfig.JournalPageSeconds) SetPhase(MuseumPhase.Journal3);
```

```
                break;
            case MuseumPhase.Journal3:
                if (_phaseTimer &gt;= NewbieConfig.JournalPageSeconds) SetPhase(MuseumPhase.Playing);
                break;
            case MuseumPhase.CardShow:
                if (_phaseTimer &gt;= NewbieConfig.CardShowSeconds) SetPhase(MuseumPhase.WonderPreview);
                break;
            case MuseumPhase.WonderPreview:
                //          0.40→0.75
                float k = Math.Min(1f, _phaseTimer / NewbieConfig.WonderPreviewSeconds);
                RegionProgress = Lerp(NewbieConfig.RegionProgressStart, NewbieConfig.RegionProgressEnd, k);
                if (_phaseTimer &gt;= NewbieConfig.WonderPreviewSeconds)
                {
                    Settled?.Invoke();
                    SetPhase(MuseumPhase.Settled);
                }
                break;
        }
    }
    static float Lerp(float a, float b, float t) =&gt; a + (b - a) * t;
    /// &lt;summary&gt;M1           -1       ×2     =  →  →    &lt;/summary&gt;
    public bool TryDig()
    {
        if (Phase != MuseumPhase.Playing) return false;
        if (Energy &lt;= 0) return false;
        for (int n = 0; n &lt; NewbieConfig.ShardsPerDig; n++)
        {
            var cell = PickEmptyCell();
            if (cell == null) return false;
            _items.Add(new ItemInst { Id = _nextId++, TypeId = NewbieConfig.ShardTypeId, Col = cell.Value.Item1, Row
        }
        Energy--;
        DigCount++;
        EnergyChanged?.Invoke(Energy);
        Dug?.Invoke();
        return true;
    }
    (int, int)? PickEmptyCell()
    {
        // 40     +      C#
        for (int attempt = 0; attempt &lt; 40; attempt++)
        {
            int c = _rng.Next(NewbieConfig.Cols), r = _rng.Next(NewbieConfig.Rows);
            if (CellIsEmpty(c, r)) return (c, r);
        }
        for (int r = 0; r &lt; NewbieConfig.Rows; r++)
            for (int c = 0; c &lt; NewbieConfig.Cols; c++)
                if (CellIsEmpty(c, r)) return (c, r);
        return null;
    }
```

```
bool CellIsEmpty(int col, int row)
{
    for (int i = 0; i &lt; _items.Count; i++)
        if (_items[i].Col == col &amp;amp;&amp;amp; _items[i].Row == row) return false;
    return true;
}
public ItemInst ItemAt(int col, int row)
{
    for (int i = 0; i &lt; _items.Count; i++)
        if (_items[i].Col == col &amp;amp;&amp;amp; _items[i].Row == row) return _items[i];
    return null;
}
public bool IsSelected(ItemInst item) =&gt; item != null &amp;amp;&amp;amp; item.Id == _selectedId;
/// &lt;summary&gt;
///   /       2 1  ·   §4.2①"      "   + =    + =
///              =        =
/// &lt;/summary&gt;
public bool SelectOrMerge(int col, int row)
{
    if (Phase != MuseumPhase.Playing) return false;
    var target = ItemAt(col, row);
    if (target == null) return false;
    if (_selectedId == -1)
    {
        _selectedId = target.Id;
        return true;   //
    }
    if (_selectedId == target.Id)
    {
        _selectedId = -1;   //     =
        return true;
    }
    var selected = FindItem(_selectedId);
    if (selected == null) { _selectedId = -1; return false; }
    if (selected.TypeId != target.TypeId || selected.TypeId &gt;= NewbieConfig.BronzeTypeId)
    {
        //        ——
        _selectedId = target.Id;
        return true;
    }
    //   2 1     Type+1  →   →
    _items.Remove(selected);
    _items.Remove(target);
    var product = new ItemInst { Id = _nextId++, TypeId = selected.TypeId + 1, Col = target.Col, Row = target.Ro
    _items.Add(product);
    _selectedId = -1;
    MergeCount++;
    if (selected.TypeId == NewbieConfig.ShardTypeId)
    {
        FragmentCount++;
```

```
        }
        else
        {
            //  + =              ——
            FragmentCount -= 2;
            BronzeCount++;
            MisMergeCount++;
            MisMerged?.Invoke(product);
        }
        Merged?.Invoke(selected, target, product);
        return true;
    }
    ItemInst FindItem(int id)
    {
        for (int i = 0; i &lt; _items.Count; i++)
            if (_items[i].Id == id) return _items[i];
        return null;
    }
    /// &lt;summary&gt;
    ///          §4.2②             ——       2     1
    /// &lt;/summary&gt;
    public bool DecomposeBronze()
    {
        if (Phase != MuseumPhase.Playing) return false;
        if (SealCount &lt;= 0) return false;
        var bronze = FindFirstBronze();
        if (bronze == null) return false;
        _items.Remove(bronze);
        BronzeCount--;
        for (int n = 0; n &lt; 2; n++)
        {
            var cell = PickEmptyCell();
            if (cell == null) return false;
            _items.Add(new ItemInst { Id = _nextId++, TypeId = NewbieConfig.FragmentTypeId, Col = cell.Value.Item1,
            FragmentCount++;
        }
        SealCount--;
        DecomposeCount++;
        SealChanged?.Invoke(SealCount);
        Decomposed?.Invoke(bronze);
        return true;
    }
    ItemInst FindFirstBronze()
    {
        for (int i = 0; i &lt; _items.Count; i++)
            if (_items[i].TypeId == NewbieConfig.BronzeTypeId) return _items[i];
        return null;
    }
    /// &lt;summary&gt;M3      3  →     &lt;/summary&gt;
    public bool DeliverOrder()
```

```
        {
            if (Phase != MuseumPhase.Playing) return false;
            if (FragmentCount &lt; NewbieConfig.OrderFragments) return false;
            OrderDelivered = true;
            for (int i = 0; i &lt; NewbieConfig.OrderFragments; i++)
            {
                var frag = FindFirstFragment();
                if (frag != null) _items.Remove(frag);
            }
            FragmentCount -= NewbieConfig.OrderFragments;
            OrderDeliveredDone?.Invoke();
            SetPhase(MuseumPhase.CardShow);
            return true;
        }
        ItemInst FindFirstFragment()
        {
            for (int i = 0; i &lt; _items.Count; i++)
                if (_items[i].TypeId == NewbieConfig.FragmentTypeId) return _items[i];
            return null;
        }
        void SetPhase(MuseumPhase p)
        {
            if (Phase == p) return;
            Phase = p;
            _phaseTimer = 0f;
            switch (p)
            {
                case MuseumPhase.CardShow:
                    CardShown?.Invoke();
                    break;
                case MuseumPhase.WonderPreview:
                    WonderUnlocked = true;
                    RelayUnlocked = true;
                    WonderPreviewShown?.Invoke();
                    break;
            }
            PhaseChanged?.Invoke(p);
        }
    }
}
// MuseumView.cs
using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
namespace Sy.P07.Newbie
{
    /// &lt;summary&gt;
    ///                UGUI——   ×   ×   ×   ×
```

```
///   UI Initialize          YAML      =      /       →  →   /
///     +        /        /         /     /   +NPC    /
///       +      /
///                  =
/// &lt;/summary&gt;
public sealed class MuseumView : MonoBehaviour
{
    const float CellW = 150f, CellH = 150f, Gap = 8f;
    Canvas _canvas;
    Sprite _square;
    RectTransform _board;
    Text _journalText, _energyText, _fragmentText, _sealText, _hintText, _progressText;
    RectTransform _journalOverlay, _cardOverlay, _wonderOverlay, _settlePanel;
    Button _digButton, _deliverButton, _relayButton, _decomposeButton;
    Image _digImage, _deliverImage, _decomposeImage;
    Image _progressFill;
    readonly Dictionary&lt;int, ItemVisual&gt; _items = new();
    readonly List&lt;int&gt; _scratch = new();
    MuseumSim.MuseumPhase _lastPhase;
    public event Action&lt;int, int&gt; CellTapped;
    public event Action DigRequested;
    public event Action DeliverRequested;
    public event Action DecomposeRequested;
    public event Action RelayTapped;
    static readonly Color DeepSeaBlue = new Color(0.039f, 0.141f, 0.388f);   // #0A2463
    static readonly Color CoralOrange = new Color(0.957f, 0.376f, 0.212f);  // #F46036
    static readonly Color PearlWhite = new Color(0.910f, 0.910f, 0.910f);   // #E8E8E8
    static readonly Color Verdigris = new Color(0.243f, 0.533f, 0.357f);    // #3E885B
    static readonly Color AbyssBlack = new Color(0.020f, 0.078f, 0.122f);  // #05141F
    public void Initialize(Canvas canvas, MuseumSim sim)
    {
        _canvas = canvas;
        _square = MakeSquareSprite();
        BuildScene();
        BuildHud();
        BuildJournalOverlay();
        BuildCardOverlay();
        BuildWonderOverlay();
        BuildSettlePanel();
        SetHint("          ……");
    }
    // ————    ————
    void BuildScene()
    {
        var bg = MakeCanvasChild("Bg", AbyssBlack);
        bg.anchorMin = Vector2.zero;
        bg.anchorMax = Vector2.one;
        bg.anchoredPosition = Vector2.zero;
        bg.sizeDelta = Vector2.zero;
        // X027 generated art: swap the solid Bg for the deep-sea painting when present.
```

```
    var bgImg = bg.GetComponent&lt;Image&gt;();
    // 17  §1.2: L0       ——             U184
    bgImg.raycastTarget = false;
    var bgArt = Sy.Core.ArtRuntime.Get("P07_scene_01");
    if (bgArt != null)
    {
        bgImg.sprite = bgArt;
        bgImg.color = Color.white;
        bgImg.preserveAspect = true;
    }
    var boardGo = new GameObject("Board", typeof(RectTransform), typeof(Image));
    boardGo.transform.SetParent(_canvas.transform, false);
    _board = (RectTransform)boardGo.transform;
    _board.anchorMin = _board.anchorMax = new Vector2(0.5f, 0.5f);
    _board.anchoredPosition = new Vector2(0, 100);
    _board.sizeDelta = new Vector2(NewbieConfig.Cols * (CellW + Gap), NewbieConfig.Rows * (CellH + Gap));
    var boardImg = _board.GetComponent&lt;Image&gt;();
    boardImg.sprite = _square;
    boardImg.color = new Color(DeepSeaBlue.r, DeepSeaBlue.g, DeepSeaBlue.b, 0.35f);
    for (int r = 0; r &lt; NewbieConfig.Rows; r++)
    {
        for (int c = 0; c &lt; NewbieConfig.Cols; c++)
        {
            int col = c, row = r;
            var cellGo = new GameObject($"Cell_{c}_{r}", typeof(RectTransform), typeof(Image), typeof(Button));
            cellGo.transform.SetParent(_board, false);
            var rt = (RectTransform)cellGo.transform;
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
            rt.anchoredPosition = CellCenter(c, r);
            rt.sizeDelta = new Vector2(CellW, CellH);
            var img = cellGo.GetComponent&lt;Image&gt;();
            img.sprite = _square;
            img.color = new Color(0.03f, 0.10f, 0.25f, 0.9f);
            cellGo.GetComponent&lt;Button&gt;().onClick.AddListener(() =&gt; CellTapped?.Invoke(col, row));
        }
    }
}
void BuildHud()
{
    var root = _canvas.transform;
    _energyText = MakeText(root, "   " + NewbieConfig.StartEnergy, 46, new Vector2(-370, 880), TextAnchor.UpperL
    _energyText.color = CoralOrange;
    _fragmentText = MakeText(root, "   0/" + NewbieConfig.OrderFragments, 46, new Vector2(0, 880), TextAnchor.Up
    _fragmentText.color = PearlWhite;
    _progressText = MakeText(root, "     40%", 44, new Vector2(0, 820), TextAnchor.UpperCenter, 500, 60);
    _progressText.color = Verdigris;
    _sealText = MakeText(root, "    ×" + NewbieConfig.StartSeals, 42, new Vector2(350, 820), TextAnchor.UpperRig
    _sealText.color = CoralOrange;
    var barGo = new GameObject("Progress", typeof(RectTransform), typeof(Image));
    barGo.transform.SetParent(root, false);
```

```
    var barRt = (RectTransform)barGo.transform;
    barRt.anchorMin = barRt.anchorMax = new Vector2(0.5f, 0.5f);
    barRt.anchoredPosition = new Vector2(0, 760);
    barRt.sizeDelta = new Vector2(700, 18);
    var barImg = barGo.GetComponent&lt;Image&gt;();
    barImg.sprite = _square;
    barImg.color = new Color(0.1f, 0.15f, 0.25f);
    var fillGo = new GameObject("Fill", typeof(RectTransform), typeof(Image));
    fillGo.transform.SetParent(barGo.transform, false);
    var fillRt = (RectTransform)fillGo.transform;
    fillRt.anchorMin = new Vector2(0f, 0.5f);
    fillRt.anchorMax = new Vector2(1f, 0.5f);
    fillRt.anchoredPosition = Vector2.zero;
    fillRt.sizeDelta = new Vector2(0, 18);
    _progressFill = fillGo.GetComponent&lt;Image&gt;();
    _progressFill.sprite = _square;
    _progressFill.type = Image.Type.Filled;
    _progressFill.fillMethod = Image.FillMethod.Horizontal;
    _progressFill.color = Verdigris;
    _hintText = MakeText(root, "", 40, new Vector2(0, -840), TextAnchor.MiddleCenter, 1020, 150);
    _hintText.color = new Color(0.85f, 0.88f, 0.92f);
    _digButton = MakeBottomButton("      -1 ", new Vector2(-330, 100), DeepSeaBlue, () =&gt; DigRequested?.Invok
    _decomposeButton = MakeBottomButton("   ·  ", new Vector2(0, 100), DeepSeaBlue, () =&gt; DecomposeRequested?
    _deliverButton = MakeBottomButton("        ", new Vector2(330, 100), CoralOrange, () =&gt; DeliverRequested?
}
Button MakeBottomButton(string label, Vector2 pos, Color color, Action onClick, out Image image)
{
    var go = new GameObject(label, typeof(RectTransform), typeof(Image), typeof(Button));
    go.transform.SetParent(_canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0f);
    rt.anchoredPosition = pos;
    rt.sizeDelta = new Vector2(310, 120);
    image = go.GetComponent&lt;Image&gt;();
    image.sprite = _square;
    image.color = color;
    // 17  § .7: pick the label color by background luminance (WCAG
    // relative-luma) so light CTA backgrounds always carry dark text.
    float lum = color.r * 0.2126f + color.g * 0.7152f + color.b * 0.0722f;
    MakeText(go.transform, label, 34, Vector2.zero, TextAnchor.MiddleCenter, 310, 120).color = lum &gt; 0.45f ?
    var button = go.GetComponent&lt;Button&gt;();
    button.interactable = false;
    button.onClick.AddListener(() =&gt; onClick());
    return button;
}
void BuildJournalOverlay()
{
    _journalOverlay = MakeOverlayPanel("JournalOverlay", new Color(0.012f, 0.045f, 0.07f, 0.96f));
    _journalText = MakeText(_journalOverlay, "", 64, new Vector2(0, 0), TextAnchor.MiddleCenter, 1000, 200);
    _journalText.color = PearlWhite;
```

```
}
void BuildCardOverlay()
{
    _cardOverlay = MakeOverlayPanel("CardOverlay", new Color(0.012f, 0.045f, 0.07f, 0.94f));
    MakeText(_cardOverlay, "    #1", 76, new Vector2(0, 330), TextAnchor.MiddleCenter, 1000, 100).color = CoralO
    MakeText(_cardOverlay, NewbieConfig.CardText, 48, new Vector2(0, 140), TextAnchor.MiddleCenter, 1000, 120).c
    // NPC        §4.2④           ——
    MakeText(_cardOverlay, "&lt;size=36&gt;" + NewbieConfig.NpcJournalText + "&lt;/size&gt;", 40, new Vector2(0,
    _cardOverlay.gameObject.SetActive(false);
}
void BuildWonderOverlay()
{
    _wonderOverlay = MakeOverlayPanel("WonderOverlay", new Color(0, 0, 0, 0f));
    _wonderOverlay.GetComponent&lt;Image&gt;().color = new Color(0, 0, 0, 0.55f);   // 17  § .6      0.5 U184
    var silhouette = MakeChildOf(_wonderOverlay, "Silhouette", new Vector2(0, 420), new Vector2(360, 360));
    var silImg = silhouette.GetComponent&lt;Image&gt;();
    silImg.sprite = _square;
    silImg.color = new Color(DeepSeaBlue.r, DeepSeaBlue.g, DeepSeaBlue.b, 0.9f);
    MakeText(silhouette, "  ", 90, Vector2.zero, TextAnchor.MiddleCenter, 360, 360).color = new Color(PearlWhite
    MakeText(_wonderOverlay, "       ·       ", 56, new Vector2(0, 160), TextAnchor.MiddleCenter, 1000, 90).colo
    var relayGo = new GameObject("RelayButton", typeof(RectTransform), typeof(Image), typeof(Button));
    relayGo.transform.SetParent(_wonderOverlay, false);
    var relayRt = (RectTransform)relayGo.transform;
    relayRt.anchorMin = relayRt.anchorMax = new Vector2(0.5f, 0.5f);
    relayRt.anchoredPosition = new Vector2(0, -60);
    relayRt.sizeDelta = new Vector2(460, 120);
    var relayImg = relayGo.GetComponent&lt;Image&gt;();
    relayImg.sprite = _square;
    relayImg.color = CoralOrange;
    MakeText(relayGo.transform, "     ·    1    ", 38, Vector2.zero, TextAnchor.MiddleCenter, 460, 120).color =
    _relayButton = relayGo.GetComponent&lt;Button&gt;();
    _relayButton.onClick.AddListener(() =&gt; RelayTapped?.Invoke());
    _wonderOverlay.gameObject.SetActive(false);
}
void BuildSettlePanel()
{
    _settlePanel = MakeOverlayPanel("SettlePanel", new Color(0.012f, 0.045f, 0.07f, 0.96f));
    MakeText(_settlePanel, "    ·     ", 90, new Vector2(0, 560), TextAnchor.MiddleCenter, 1040, 110).color = Ve
    MakeText(_settlePanel, $"     1   ·    4      1  ·       1    2  ·     1  \n&lt;size=40&gt;            ——
    MakeActionButton(_settlePanel, "    ", new Vector2(-260, -260), () =&gt; UnityEngine.SceneManagement.SceneMa
    MakeActionButton(_settlePanel, "     ", new Vector2(260, -260), () =&gt; UnityEngine.SceneManagement.SceneMa
    _settlePanel.gameObject.SetActive(false);
}
// ————      ————
public void SyncView(MuseumSim sim)
{
    ReconcileItems(sim);
    foreach (var kv in _items)
    {
        var v = kv.Value;
```

```
            bool selected = sim.IsSelected(v.Source);
            if (selected != v.LastSelected)
            {
                v.LastSelected = selected;
                v.Rt.localScale = selected ? Vector3.one * 1.15f : Vector3.one;
            }
        }
        float progress = sim.RegionProgress;
        _progressFill.fillAmount = progress;
        int pct = (int)(progress * 100f);
        if (_lastProgressPct != pct) { _lastProgressPct = pct; _progressText.text = "     " + pct + "%"; }
        bool playing = sim.Phase == MuseumSim.MuseumPhase.Playing;
        bool digOk = playing &amp;&amp; sim.Energy &gt; 0;
        if (_digButton.interactable != digOk)
        {
            _digButton.interactable = digOk;
            _digImage.color = digOk ? DeepSeaBlue : new Color(0.2f, 0.22f, 0.3f);
        }
        bool decomposeOk = playing &amp;&amp; sim.BronzeCount &gt; 0 &amp;&amp; sim.SealCount &gt; 0;
        if (_decomposeButton.interactable != decomposeOk)
        {
            _decomposeButton.interactable = decomposeOk;
            _decomposeImage.color = decomposeOk ? CoralOrange : new Color(0.2f, 0.22f, 0.3f);
        }
        bool deliverOk = playing &amp;&amp; sim.FragmentCount &gt;= NewbieConfig.OrderFragments;
        if (_deliverButton.interactable != deliverOk)
        {
            _deliverButton.interactable = deliverOk;
            _deliverImage.color = deliverOk ? CoralOrange : new Color(0.2f, 0.22f, 0.3f);
        }
        if (sim.Phase != _lastPhase)
        {
            _lastPhase = sim.Phase;
            OnPhaseVisual(sim.Phase);
        }
    }
    int _lastProgressPct = -1;
    void OnPhaseVisual(MuseumSim.MuseumPhase phase)
    {
        switch (phase)
        {
            case MuseumSim.MuseumPhase.Journal1:
                _journalOverlay.gameObject.SetActive(true);
                _journalText.text = "     ·  1 \n&lt;size=40&gt;      7             &lt;/size&gt;";
                break;
            case MuseumSim.MuseumPhase.Journal2:
                _journalText.text = "         \n&lt;size=40&gt;——            &lt;/size&gt;";
                break;
            case MuseumSim.MuseumPhase.Journal3:
                _journalText.text = "      \n&lt;size=40&gt;           &lt;/size&gt;";
```

```
            break;
        case MuseumSim.MuseumPhase.Playing:
            _journalOverlay.gameObject.SetActive(false);
            break;
        case MuseumSim.MuseumPhase.CardShow:
            _cardOverlay.gameObject.SetActive(true);
            break;
        case MuseumSim.MuseumPhase.WonderPreview:
            _cardOverlay.gameObject.SetActive(false);
            _wonderOverlay.gameObject.SetActive(true);
            StartCoroutine(RelayBreathingRoutine());
            break;
        case MuseumSim.MuseumPhase.Settled:
            _wonderOverlay.gameObject.SetActive(false);
            _settlePanel.gameObject.SetActive(true);
            break;
    }
}
/// &lt;summary&gt;             ——M5         &lt;/summary&gt;
IEnumerator RelayBreathingRoutine()
{
    while (_wonderOverlay.gameObject.activeSelf)
    {
        float k = 0.5f + 0.5f * Mathf.Sin(Time.unscaledTime * 3f);
        _relayButton.transform.localScale = Vector3.one * (1f + 0.05f * k);
        yield return null;
    }
    _relayButton.transform.localScale = Vector3.one;
}
void ReconcileItems(MuseumSim sim)
{
    _scratch.Clear();
    foreach (var kv in _items)
        if (!ContainsItem(sim, kv.Key)) _scratch.Add(kv.Key);
    for (int i = 0; i &lt; _scratch.Count; i++)
    {
        Destroy(_items[_scratch[i]].Go);
        _items.Remove(_scratch[i]);
    }
    for (int i = 0; i &lt; sim.Items.Count; i++)
    {
        var item = sim.Items[i];
        if (!_items.TryGetValue(item.Id, out var v))
        {
            v = CreateItemVisual(item);
            _items[item.Id] = v;
        }
        v.Source = item;
    }
}
```

```
static bool ContainsItem(MuseumSim sim, int id)
{
    for (int i = 0; i &lt; sim.Items.Count; i++)
        if (sim.Items[i].Id == id) return true;
    return false;
}
ItemVisual CreateItemVisual(MuseumSim.ItemInst item)
{
    var go = new GameObject("Item_" + item.Id, typeof(RectTransform), typeof(Image));
    go.transform.SetParent(_board, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = CellCenter(item.Col, item.Row);
    rt.sizeDelta = new Vector2(120, 120);
    var img = go.GetComponent&lt;Image&gt;();
    img.sprite = _square;
    string label;
    Color labelColor;
    switch (item.TypeId)
    {
        case NewbieConfig.BronzeTypeId:   //            ——
            img.color = CoralOrange;
            label = " ";
            labelColor = AbyssBlack;
            break;
        case NewbieConfig.FragmentTypeId:
            img.color = Verdigris;
            label = " ";
            labelColor = PearlWhite;
            break;
        default:
        {   //        ——X027 generated art when present
            var relicArt = Sy.Core.ArtRuntime.Get("P07_unit_01");
            if (relicArt != null)
            {
                img.sprite = relicArt;
                img.color = Color.white;
                img.preserveAspect = true;
            }
            else img.color = PearlWhite;
            label = " ";
            labelColor = AbyssBlack;
            break;
        }
    }
    MakeText(go.transform, label, 52, Vector2.zero, TextAnchor.MiddleCenter, 120, 120).color = labelColor;
    return new ItemVisual { Go = go, Rt = rt, Source = item };
}
// ————    ————
/// &lt;summary&gt;        §        →  →   ——         &lt;/summary&gt;
```

```
public void PlayDigFx()
{
    StartCoroutine(DigFramesRoutine());
}
IEnumerator DigFramesRoutine()
{
    var go = new GameObject("DigFrames", typeof(RectTransform), typeof(Text));
    go.transform.SetParent(_canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = new Vector2(0, 300);
    rt.sizeDelta = new Vector2(600, 100);
    var t = go.GetComponent&lt;Text&gt;();
    t.font = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
    t.fontSize = 56;
    t.alignment = TextAnchor.MiddleCenter;
    t.color = CoralOrange;
    for (int i = 0; i &lt; NewbieConfig.ExcavateStages.Length; i++)
    {
        t.text = NewbieConfig.ExcavateStages[i];
        yield return new WaitForSeconds(NewbieConfig.ExcavateFrameSeconds);
    }
    Destroy(go);
}
/// &lt;summary&gt;          +       ——T006      &lt;/summary&gt;
public void PlayMergeFx(int itemId)
{
    if (_items.TryGetValue(itemId, out var v))
        StartCoroutine(BounceRoutine(v.Rt));
    StartCoroutine(FlashRoutine(new Color(1f, 1f, 1f, 0.35f)));
}
/// &lt;summary&gt;              "         "  &lt;/summary&gt;
public void PlayDecomposeFx()
{
    StartCoroutine(FlashRoutine(new Color(Verdigris.r, Verdigris.g, Verdigris.b, 0.45f)));
}
IEnumerator BounceRoutine(RectTransform rt)
{
    float t = 0f;
    while (t &lt; NewbieConfig.MergeBounceSeconds)
    {
        t += Time.deltaTime;
        float k = Mathf.Clamp01(t / NewbieConfig.MergeBounceSeconds);
        rt.localScale = Vector3.one * Mathf.Lerp(1.45f, 1f, k);
        yield return null;
    }
    rt.localScale = Vector3.one;
}
IEnumerator FlashRoutine(Color color)
{
```

```
    var go = new GameObject("MergeFlash", typeof(RectTransform), typeof(Image));
    go.transform.SetParent(_canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = Vector2.zero;
    rt.anchorMax = Vector2.one;
    rt.anchoredPosition = Vector2.zero;
    rt.sizeDelta = Vector2.zero;
    var img = go.GetComponent&lt;Image&gt;();
    img.sprite = _square;
    img.color = color;
    img.raycastTarget = false;   // 17  §1.2         U184
    yield return new WaitForSeconds(NewbieConfig.MergeFlashSeconds);
    Destroy(go);
}
// ——— HUD    ———
public void SetEnergy(int e) =&gt; _energyText.text = "   " + e;
public void SetFragments(int n) =&gt; _fragmentText.text = "   " + n + "/" + NewbieConfig.OrderFragments;
public void SetSeals(int s) =&gt; _sealText.text = "    ×" + s;
public void SetHint(string hint) =&gt; _hintText.text = hint;
// ———    ———
static Vector2 CellCenter(int col, int row)
{
    float x = (col - (NewbieConfig.Cols - 1) * 0.5f) * (CellW + Gap);
    float y = ((NewbieConfig.Rows - 1) * 0.5f - row) * (CellH + Gap);
    return new Vector2(x, y);
}
RectTransform MakeCanvasChild(string name, Color color)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image));
    go.transform.SetParent(_canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    var img = rt.GetComponent&lt;Image&gt;();
    img.sprite = _square;
    img.color = color;
    return rt;
}
RectTransform MakeChildOf(RectTransform parent, string name, Vector2 pos, Vector2 size)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = pos;
    rt.sizeDelta = size;
    return rt;
}
RectTransform MakeOverlayPanel(string name, Color color)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image));
```

```
        go.transform.SetParent(_canvas.transform, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = Vector2.zero;
        rt.anchorMax = Vector2.one;
        rt.offsetMin = rt.offsetMax = Vector2.zero;
        var image = go.GetComponent&lt;Image&gt;();
        image.sprite = _square;
        image.color = color;
        return rt;
    }
    void MakeActionButton(RectTransform parent, string text, Vector2 pos, Action onClick)
    {
        var go = new GameObject(text, typeof(RectTransform), typeof(Image), typeof(Button));
        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = pos;
        rt.sizeDelta = new Vector2(420, 120);
        var image = go.GetComponent&lt;Image&gt;();
        image.sprite = _square;
        image.color = Verdigris;
        MakeText(go.transform, text, 44, Vector2.zero, TextAnchor.MiddleCenter, 420, 120).color = PearlWhite;
        go.GetComponent&lt;Button&gt;().onClick.AddListener(() =&gt; onClick());
    }
    static Text MakeText(Transform parent, string text, int size, Vector2 pos, TextAnchor anchor, float w, float h)
    {
        var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = pos;
        rt.sizeDelta = new Vector2(w, h);
        var t = go.GetComponent&lt;Text&gt;();
        t.font = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
        t.text = text;
        t.fontSize = size;
        t.alignment = anchor;
        t.color = new Color(0.91f, 0.89f, 0.85f);
        t.supportRichText = true;
        t.raycastTarget = false;   // 17  §1.2          U184
        return t;
    }
    static Sprite MakeSquareSprite()
    {
        var tex = new Texture2D(8, 8, TextureFormat.RGBA32, false);
        var px = tex.GetPixels32();
        for (int i = 0; i &lt; px.Length; i++) px[i] = new Color32(255, 255, 255, 255);
        tex.SetPixels32(px);
        tex.Apply(false, false);
        return Sprite.Create(tex, new Rect(0, 0, 8, 8), new Vector2(0.5f, 0.5f), 8f);
```

```
        }
        // ————        ————
        sealed class ItemVisual
        {
            public GameObject Go;
            public RectTransform Rt;
            public MuseumSim.ItemInst Source;
            public bool LastSelected;
        }
    }
}
// NewbieConfig.cs
namespace Sy.P07.Newbie
{
    /// &lt;summary&gt;
    /// P07         ·          V0.2   ——            X009+IConfigService      Design/configs/P07 JSON
    ///        §4.2①    +   +NPC  /②       ="         "[    1 ·    3 ·          ]/
    /// ③             =      06    07 M0-M5
    ///       #0A2463 ×    #F46036 ×    #E8E8E8 ×    #3E885B ×    #05141F
    ///         3       =3 ×2  =6  →3   =3       = +             →
    /// &lt;/summary&gt;
    public static class NewbieConfig
    {
        public const string GameTitle = "     ";
        public const string GameSlogan = "            ";
        public const int Cols = 6;                 // 6×7
        public const int Rows = 7;
        public const int StartEnergy = 3;          //            1    {regenSec:240}——   2-3
        public const int ShardsPerDig = 2;        //             3 =6  =3  =  3
        public const int OrderFragments = 3;      //        ×3
        // ——        §4.2①           ——
        public const int ShardTypeId = 0;          //
        public const int FragmentTypeId = 1;       //        ·    =
        public const int BronzeTypeId = 2;         //       +   ——            12 ×6
        public static readonly string[] ItemNames = { "  ", "  ", "    " };
        // ——            §4.2②              ——
        public const int StartSeals = 3;           //       3          1
        public const string SealRuleNote = "          1 ·                   ";
        // ——         §    →  →        ——
        public static readonly string[] ExcavateStages = { "  ……", "   ", "  ——   " };
        // ——    +NPC        §4.2①           ——       ——
        public const string CardText = "                …… ";
        public const string NpcJournalText =
            "     .                              ——              ";
        public const float JournalPageSeconds = 0.8f;   // M0    3   2=   / 3=     ——U184    3 ×0.8=2.4s&lt;  3s   17  §
        public const float CardShowSeconds = 1.6f;      // M4   +NPC
        public const float WonderPreviewSeconds = 1.6f; // M5       0.40→0.75
        public const float MergeBounceSeconds = 0.2f;    //
        public const float MergeFlashSeconds = 0.1f;    //        =
        public const float ExcavateFrameSeconds = 0.15f; //
```

```
        public const float MaxTutorialSeconds = 210f;   //      ≤3.5min 06  ②
        public const float RegionProgressStart = 0.40f;
        public const float RegionProgressEnd = 0.75f;
        public const int SimSeed = 20260916;
    }
}
// TutorialFlow.cs
using System;
namespace Sy.P07.Newbie
{
    /// &lt;summary&gt;
    ///        evt_newbie_step 1-5 ①  ②   ③   ④   ⑤   +
    ///      MuseumSim   3  /      /              /  /    ——
    /// Play   headless   NewbieFlowCheck
    /// &lt;/summary&gt;
    public sealed class TutorialFlow
    {
        public int Step { get; private set; }
        readonly Action&lt;string&gt; _hint;
        readonly Action&lt;int&gt; _onStep;
        bool _firstDigSeen;
        bool _firstMergeSeen;
        public TutorialFlow(MuseumSim sim, Action&lt;string&gt; hint, Action&lt;int&gt; onStep)
        {
            _hint = hint ?? delegate { };
            _onStep = onStep ?? delegate { };
            sim.Dug += OnDug;
            sim.Merged += OnMerged;
            sim.MisMerged += OnMisMerged;
            sim.Decomposed += OnDecomposed;
            sim.OrderDeliveredDone += OnOrderDelivered;
            sim.CardShown += OnCardShown;
            sim.Settled += OnSettled;
            sim.PhaseChanged += OnPhaseChanged;
            _hint("      ——          ");
        }
        void Advance(int step)
        {
            if (Step &gt;= step) return;
            Step = step;
            _onStep(step);
        }
        void OnDug()
        {
            if (!_firstDigSeen)
            {
                _firstDigSeen = true;
                Advance(1);
                _hint("      →  →     ×2   ——  -1         ");
                return;
```

```
    }
    _hint("     ×2   →  →   ——               ");
}
void OnMerged(MuseumSim.ItemInst a, MuseumSim.ItemInst b, MuseumSim.ItemInst product)
{
    if (product.TypeId == NewbieConfig.BronzeTypeId) return;   //    OnMisMerged
    if (!_firstMergeSeen)
    {
        _firstMergeSeen = true;
        Advance(2);
        _hint("      +  =  ——  +             ");
        return;
    }
    _hint(NewbieConfig.ItemNames[product.TypeId] + "+1——        " + NewbieConfig.OrderFragments + "   ");
}
void OnMisMerged(MuseumSim.ItemInst bronze)
{
    _hint("      +      " + NewbieConfig.ItemNames[NewbieConfig.BronzeTypeId]
          + " ——            " + NewbieConfig.SealRuleNote);
}
void OnDecomposed(MuseumSim.ItemInst bronze)
{
    _hint("             2   ——                     -1 ");
}
void OnOrderDelivered()
{
    Advance(3);
    _hint("        ——             ");
}
void OnCardShown()
{
    Advance(4);
    _hint("   #1  ——" + NewbieConfig.CardText + "\n" + NewbieConfig.NpcJournalText);
}
void OnSettled()
{
    Advance(5);
    _hint("                   ——           ");
}
void OnPhaseChanged(MuseumSim.MuseumPhase phase)
{
    switch (phase)
    {
        case MuseumSim.MuseumPhase.Journal2:
            _hint("         ——           ");
            break;
        case MuseumSim.MuseumPhase.Journal3:
            _hint("      ——          ");
            break;
        case MuseumSim.MuseumPhase.Playing:
```

```
                    _hint("                    3    -1 ");
                    break;
                case MuseumSim.MuseumPhase.WonderPreview:
                    _hint("    ——          1         ");
                    break;
            }
        }
    }
}
```